BINF 6210/8210:
Machine Learning for Bioinformatics

Decision Trees

Outline

Introduction to Decision Trees

Decision Trees for Classification

Decision Trees for Regression

Decision Trees Overfitting and Pruning

Learning Objectives

- Understand how decision trees make predictions
- Describe recursive tree induction and split selection
- Compare classification and regression trees
- Explain overfitting, complexity control, and pruning
- Implement decision trees in Python

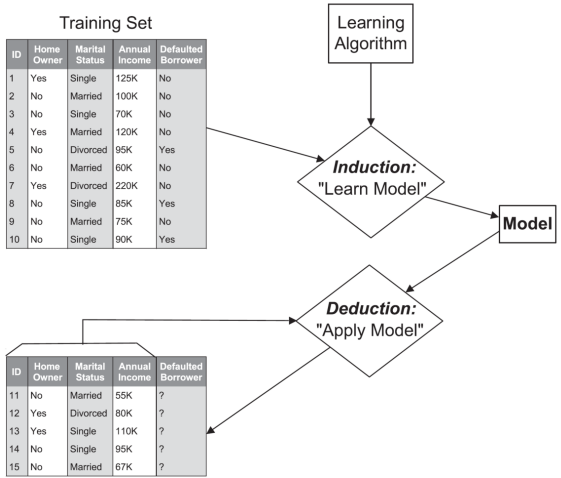
Sources and Acknowledgments

Some original diagrams and examples are adapted from the data mining materials by Tan, Steinbach, and Kumar.

`https://www-users.cse.umn.edu/~kumar001/dmbook/index.php`

Introduction to Decision Trees

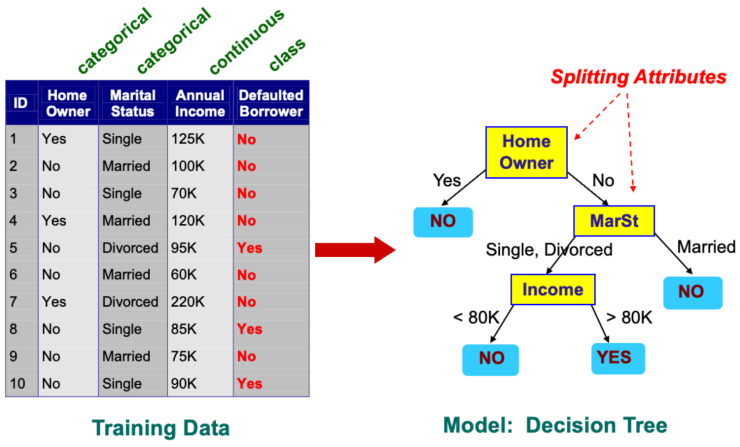
Building and Using a Predictive Model



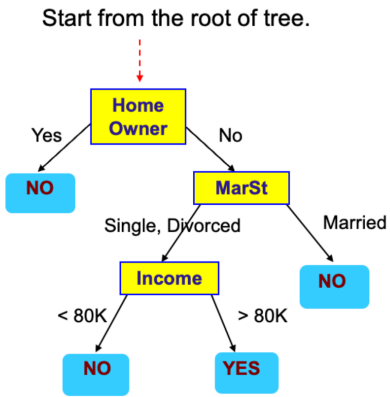
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
11	No	Married	55K	?
12	Yes	Divorced	80K	?
13	Yes	Single	110K	?
14	No	Single	95K	?
15	No	Married	67K	?

Example of a Decision Tree



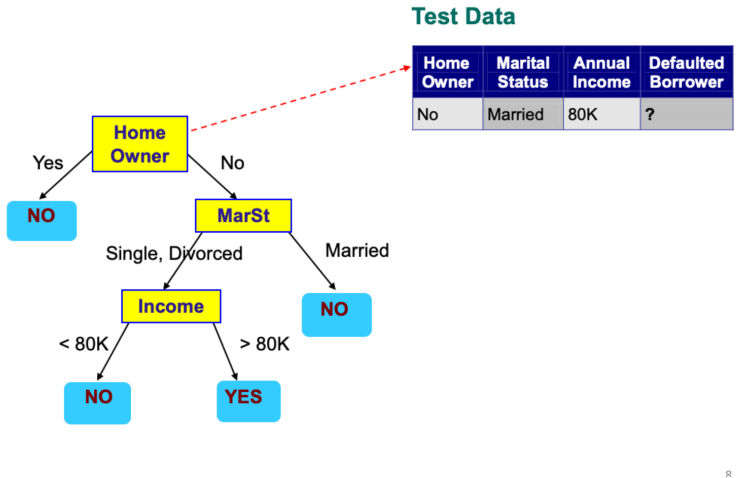
Apply Model to Test Data



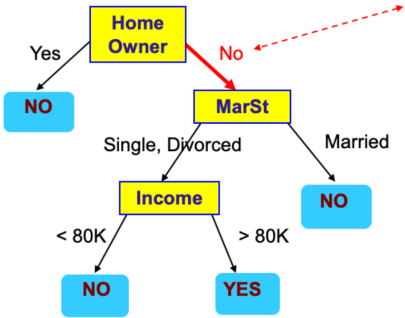
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Apply Model to Test Data



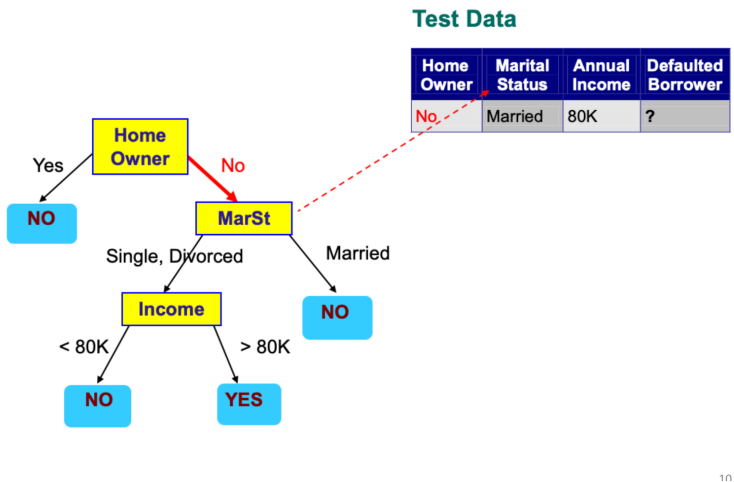
Apply Model to Test Data



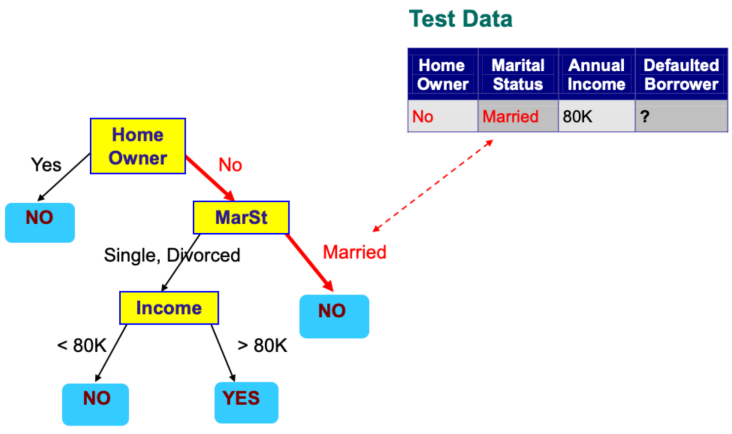
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

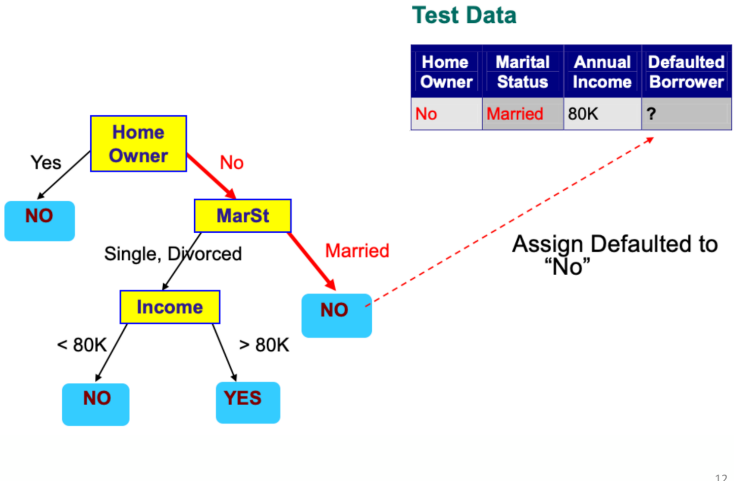
Apply Model to Test Data



Apply Model to Test Data

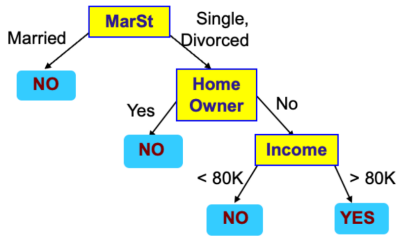


Apply Model to Test Data



Another Example of a Decision Tree

	categorical	categorical	continuous	class
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



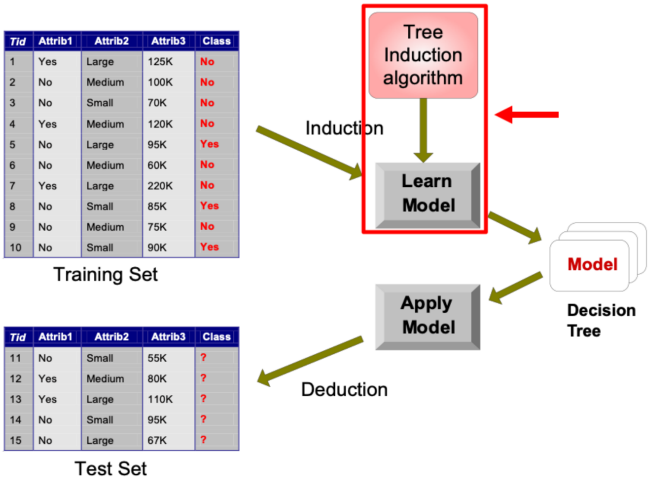
There could be more than one tree that fits the same data!

Decision Tree Core Idea

- Recursively partitions the feature space
- Uses interpretable if–then decision rules
- Predicts by following a root-to-leaf path
- Selects splits greedily to reduce impurity or squared error
- Decision trees can be used for both **classification** and **regression**.

Decision Trees for Classification

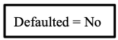
Decision Tree Classification Task



Decision Tree Induction (Hunt's Algorithm)

- Start with all training observations at the root node.
- At each node:
  - If a stopping criterion is satisfied, create a leaf node.
  - Otherwise, select a splitting condition and partition the observations into child nodes.
- Recursively apply the same procedure to each child node.

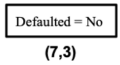
Decision Tree Induction (Hunt's Algorithm)



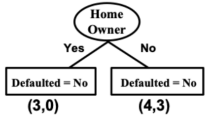
(a)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Decision Tree Induction (Hunt's Algorithm)



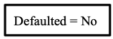
(a)



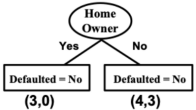
(b)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Decision Tree Induction (Hunt's Algorithm)

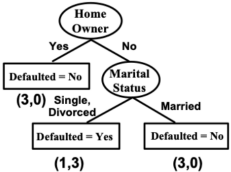


(a)



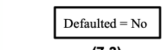
(b)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

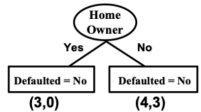


(c)

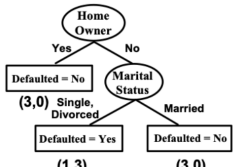
Decision Tree Induction (Hunt's Algorithm)



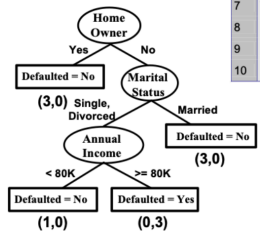
(a)



(b)



(c)



(d)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Note: The splitting conditions shown in these diagrams have already been selected.

Decision Tree Induction: Key Design Decisions

- **How should a node be split?**
  Select a feature and splitting condition that maximizes impurity reduction (classification) or squared-error reduction (regression).
- **When should splitting stop?**
  Stop when a node is sufficiently homogeneous, no valid split is available, or a complexity constraint is reached.
- **How should tree complexity be controlled?**
  - Stopping criteria: Determine when to stop splitting a node during tree construction.
  - Post-pruning: Removes branches from an already constructed tree to reduce complexity and improve generalization.

Major Decision Tree Algorithms

- **Hunt's Algorithm:** General framework for recursive decision-tree induction.
- The following are specific decision-tree algorithms that follow this general principle but differ in how they select splits and control tree complexity.
  - **ID3**
    - Information gain (entropy) for split selection.
    - Primarily categorical features; multiway splits.
  - **C4.5**
    - Extends ID3 by using gain ratio for split selection.
    - Supports continuous features and pruning.
  - **CART** (Classification and Regression Trees)
    - Binary recursive partitioning.
    - Commonly uses Gini impurity for classification and squared error for regression.
    - Basis of scikit-learn's decision trees.

CART

- CART is a widely used decision-tree methodology based on recursive binary partitioning.
- CART uses impurity-based splitting for classification and squared-error-based splitting for regression.
- CART is the primary decision-tree methodology emphasized in this lecture because it supports both classification and regression and forms the basis of scikit-learn's decision trees.

Feature Types in Decision Trees

Decision trees can use different types of input features:

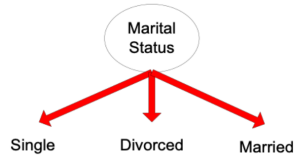
- **Binary:** Two possible values (e.g., yes/no, present/absent).
- **Nominal:** Unordered categories (e.g., sample type, blood type).
- **Ordinal:** Ordered categories (e.g., low, medium, high).
- **Continuous:** Numerical measurements (e.g., gene expression, metabolite concentration).

Key Point

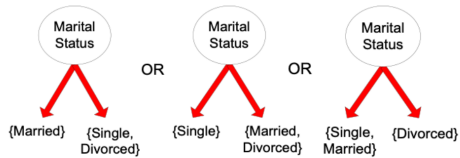
- Different feature types may require different splitting conditions.
- **CART uses binary splits**, regardless of the feature type.

Test Condition for Nominal Attributes

- Multi-way split:
  - Use as many partitions as distinct values.



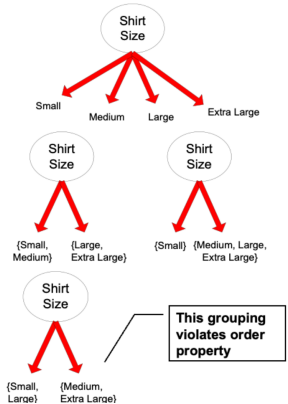
- Binary split:
  - Divides values into two subsets



Note: Multiway splitting is used by algorithms such as ID3 and C4.5. CART uses binary splits, partitioning categories into two groups.

Test Condition for Ordinal Attributes

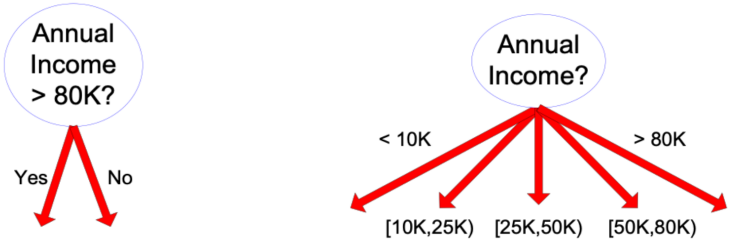
- Multi-way split:
  - Use as many partitions as distinct values
- Binary split:
  - Divides values into two subsets
  - Preserve order property among attribute values



Note: CART uses binary splits. When ordinal features are treated as ordered variables, threshold-based splits preserve the natural ordering of categories. Multiway splitting is shown for comparison.

Test Condition for Continuous Attributes

Continuous features can be partitioned using binary or multiway splits, depending on the decision-tree algorithm.



Note: CART uses threshold-based binary splits for continuous features. The threshold is selected to maximize impurity reduction (classification) or squared-error reduction (regression).

How to Determine the Best Split?

These examples illustrate candidate splitting strategies across different decision-tree algorithms; CART considers binary splits only.

Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Before Splitting: 10 records of class 0, 10 records of class 1



Which test condition is the best?

How to Determine the Best Split?

- Greedy approach:
  - Nodes with purer class distribution are preferred
- Need a measure of node impurity:

C0: 5 C1: 5	C0: 9 C1: 1
High degree of impurity	**Low degree of impurity**

Measures of Node Impurity

For a node t with c classes and class proportions $p_i(t)$:

$$\text{Gini}(t) = 1 - \sum_{i=1}^{c} p_i(t)^2$$

$$H(t) = -\sum_{i=1}^{c} p_i(t) \log_2 p_i(t)$$

By convention, $0 \log_2 0 = 0$.

Finding the Best Split

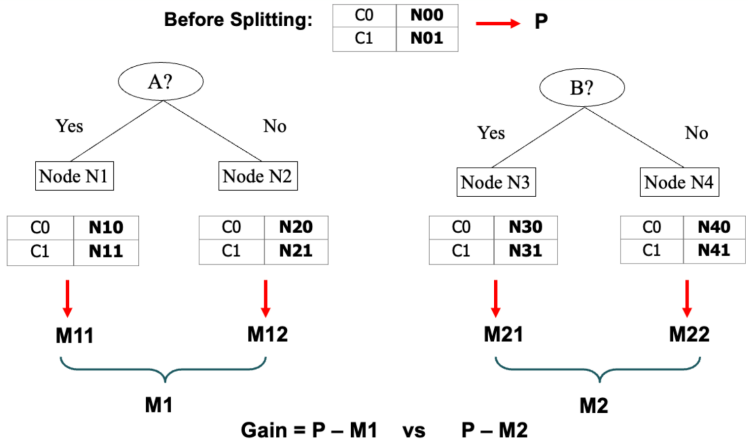
For a parent node t and children $t_1, \ldots, t_k$:

$$I_{\text{after}} = \sum_{j=1}^{k} \frac{n_j}{n_t} I(t_j)$$

$$\boxed{\text{Gain} = I(t) - I_{\text{after}}}$$

Choose the candidate split that maximizes the impurity reduction (equivalently, minimizes weighted child impurity).

Finding the Best Split



Measures of Node Impurity: Gini

$$\text{Gini}(t) = 1 - \sum_{i=1}^{c} p_i(t)^2$$

- Minimum: 0 for a pure node.
- Maximum: $1 - 1/c$ when the c classes are equally represented.
- Binary classification: Gini $= 2p(1-p)$.

Gini Index for a Collection of Nodes

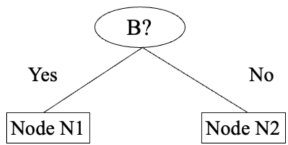
For k children of a parent containing n samples:

$$\text{Gini}_{\text{split}} = \sum_{j=1}^{k} \frac{n_j}{n} \text{Gini}(t_j)$$

- n_j is the number of samples in child j.
- The weighted average accounts for child sizes.
- Prefer the split with the smallest weighted child Gini.

Gini Index for a Collection of Nodes

- Splits into two partitions (child nodes)
- Effect of Weighing partitions:
  - Larger and purer partitions are sought



	Parent
C1	7
C2	5
Gini = 0.486	

Gini(N1)
= 1 – (5/6)² – (1/6)²
= 0.278

Gini(N2)
= 1 – (2/6)² – (4/6)²
= 0.444

	N1	N2
C1	5	2
C2	1	4
Gini=0.361		

Weighted Gini of N1 N2
= 6/12 * 0.278 +
6/12 * 0.444
= 0.361

Gain = 0.486 – 0.361 = 0.125

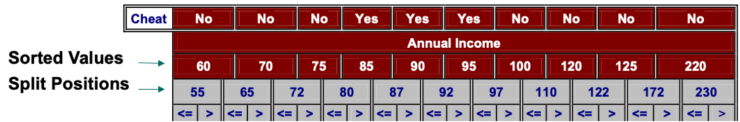
Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No
	Annual Income									
Sorted Values →	60	70	75	85	90	95	100	120	125	220

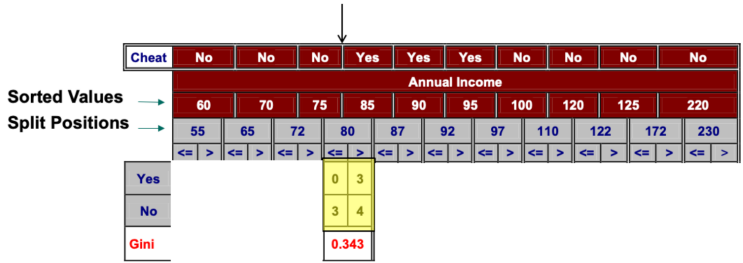
Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index



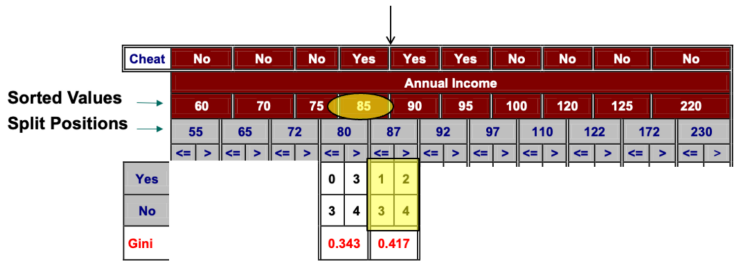
Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index



Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index



Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No
Annual Income										
Sorted Values →	60	70	75	85	90	95	100	120	125	220
Split Positions →	55	65	72	80	87	92	97	110	122	172	230

	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
Gini	0.420		0.400		0.375		0.343		0.417		0.400		***0.300***		0.343		0.375		0.400		0.420	

Measures of Node Impurity: Entropy

$$H(t) = -\sum_{i=1}^{c} p_i(t) \log_2 p_i(t)$$

- Minimum: 0 for a pure node.
- Maximum: $\log_2 c$ for equally represented classes.
- $p_i(t)$ is the *relative frequency* of class i at node t.

Entropy of a Single Node

For a node t with c classes:

$$H(t) = -\sum_{i=1}^{c} p_i(t) \log_2 p_i(t)$$

C1	0
C2	6

$$H(t) = -0 \log_2 0 - 1 \log_2 1$$
$$= \boxed{0}$$

C1	1
C2	5

$$H(t) = -\frac{1}{6} \log_2 \frac{1}{6} - \frac{5}{6} \log_2 \frac{5}{6}$$
$$\approx \boxed{0.650}$$

C1	2
C2	4

$$H(t) = -\frac{2}{6} \log_2 \frac{2}{6} - \frac{4}{6} \log_2 \frac{4}{6}$$
$$\approx \boxed{0.918}$$

Entropy increases as the class distribution becomes more balanced.

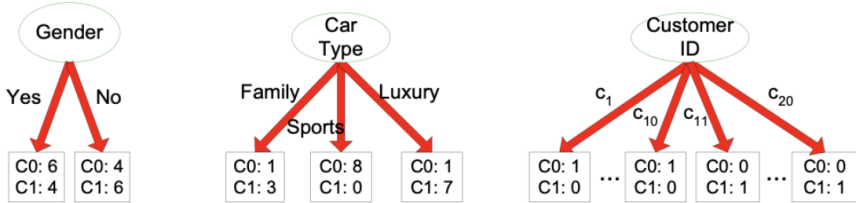
Information Gain after Splitting

$$\text{Information Gain} = H(t) - \sum_{j=1}^{k} \frac{n_j}{n_t} H(t_j)$$

- t is the parent node.
- t_j denotes child j.
- Choose the split with the greatest information gain.

Problem with Large Number of Partitions

- Information gain tends to favor attributes with many distinct values because they can produce numerous small, highly pure child nodes.



- Customer ID achieves maximum information gain because every child node is pure, but such a split is unlikely to generalize to unseen data.

Gain Ratio

- Information gain tends to favor splits that produce many child nodes.
- Gain ratio adjusts information gain by accounting for the number and sizes of child nodes.

$$\boxed{\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{Split Information}}}$$

$$\text{Split Information} = -\sum_{j=1}^{k} \frac{n_j}{n} \log_2 \left(\frac{n_j}{n} \right)$$

- k: number of child nodes.
- n_j: number of observations in child node j.
- n: number of observations in the parent node.

Key Idea

Gain ratio penalizes splits that fragment the data into many small partitions.

Decision Trees for Regression

Classification vs. Regression

Classification

- Categorical response: disease vs. no disease, species class
- Leaf predicts a class label or estimated class probabilities
- Common split criteria: Gini and entropy
- Examples: medical diagnosis and sample classification

Regression

- Continuous response: concentration, expression level, or risk score
- Leaf predicts a numerical value, usually the mean of training targets in that leaf
- Split criterion: reduction in within-node squared error or variance
- Examples: predicting metabolite concentration or environmental exposure

How Does a Regression Tree Make Predictions?

- A regression tree recursively partitions the feature space using binary splitting conditions.
- Each leaf node represents a region of the feature space and predicts a constant numerical value.
- For squared-error regression trees, the prediction at each leaf is the mean of the training target values in that leaf.
- To predict the response for a new observation:
  1. Start at the root node.
  2. Follow the splitting conditions to a leaf.
  3. Return the prediction stored in that leaf.

Regression Tree: Leaf Prediction

Key Idea

Regression trees produce **piecewise-constant predictions**: observations reaching the same leaf receive the same predicted value.

A regression-tree leaf predicts the mean of its training targets:

$$\hat{y}_{\text{leaf}} = \frac{1}{n_{\text{leaf}}} \sum_{i \in \text{leaf}} y_i$$

This constant minimizes the sum of squared errors within the leaf.

Regression Tree: Within-Node Error

For a node containing n observations:

$$\text{Var}(t) = \frac{1}{n} \sum_{i \in t} (y_i - \bar{y}_t)^2$$

$$\text{MSE}(t) = \frac{1}{n} \sum_{i \in t} (y_i - \hat{y}_t)^2$$

Since $\hat{y}_t = \bar{y}_t$, the two expressions coincide here.

Goal: minimize within-node squared error.

Regression Tree: Parent Error

Before splitting a parent node t containing n samples:

$$\bar{y}_t = \frac{1}{n} \sum_{i \in t} y_i$$

$$\text{Var}(t) = \frac{1}{n} \sum_{i \in t} (y_i - \bar{y}_t)^2$$

This measures the variation in the target before splitting.

Regression Tree: Child Errors

A candidate split creates left and right children:

$$\text{Var}_L = \frac{1}{n_L} \sum_{i \in L} (y_i - \bar{y}_L)^2$$

$$\text{Var}_R = \frac{1}{n_R} \sum_{i \in R} (y_i - \bar{y}_R)^2$$

$$\boxed{\text{Var}_{\text{children}} = \frac{n_L}{n} \text{Var}_L + \frac{n_R}{n} \text{Var}_R}$$

Regression Tree: Variance Reduction

Choose the split that maximizes:

$$\Delta \text{Var} = \text{Var}_{\text{parent}} - \left(\frac{n_L}{n} \text{Var}_L + \frac{n_R}{n} \text{Var}_R \right)$$

Equivalently, minimize the weighted variance of the child nodes.

Regression Tree: Worked Example

Parent targets: $y = [2, 3, 8, 9]$; parent mean $\bar{y} = 5.5$.

Candidate split: $L = [2, 3]$, $R = [8, 9]$.

$$\mathrm{Var}_L = 0.25, \quad \mathrm{Var}_R = 0.25$$

$$\mathrm{Var}_{\text{children}} = \frac{2}{4}(0.25) + \frac{2}{4}(0.25) = 0.25$$

$$\mathrm{Var}_{\text{parent}} = 9.25$$

$$\boxed{\Delta\,\mathrm{Var} = 9.25 - 0.25 = 9.0}$$

Decision Tree Overfitting and Pruning

Overfitting

- A tree can fit the training data extremely well while learning noise
- An overfit tree often performs worse on unseen data
- A large training–validation performance gap can indicate overfitting
- Use validation or cross-validation to select complexity; reserve the held-out test set for final evaluation

Bias–Variance Tradeoff

- **Bias:** systematic deviation of average predictions from the target function.
- **Variance:** sensitivity of predictions to the training dataset.

$$\mathbb{E}[(Y - \hat{f}(X))^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Under the standard squared-error decomposition assumptions.

Bias–Variance Tradeoff

- Shallow trees generally have higher bias and lower variance
- Deep trees generally have lower bias and higher variance
- Increasing complexity may improve training fit while harming generalization
- Use cross-validation to select an appropriate complexity level

Decision Tree Complexity

Increasing model complexity typically reduces training error but can increase variance and overfitting risk.

Tree depth and number of leaves control model capacity.

Decision Tree Complexity: Depth

- Tree depth is the number of edges along the longest root-to-leaf path
- Root node has depth 0
- Deeper trees can represent more complicated rules
- In scikit-learn: `DecisionTreeClassifier(max_depth=k)`

Decision Tree Complexity: Other Measures

- Number of leaves: each leaf defines a prediction region.
- Number of internal nodes: each adds a decision rule.
- Cost-complexity objective:

$$R_\alpha(T) = R(T) + \alpha|T|$$

where $R(T)$ is the total leaf-weighted training impurity and $|T|$ is the number of leaves.

Decision Tree Depth

- Root node is at depth 0; each edge increases depth by 1
- Greater depth permits greater model complexity
- Very deep trees can memorize training observations
- Control depth using `max_depth` and related parameters

Decision Tree Overfitting

- Greedy splitting can continue until leaves are very small
- A fully grown tree may achieve near-zero training error
- Low training error does not imply low generalization error
- Validation error often has a minimum at intermediate complexity

Diagnosing Overfitting

- Compare training and validation performance
- Use cross-validation within the development data to estimate generalization
- Look for increasing validation error as complexity grows
- Evaluate once on an untouched test set after model selection

Preventing Overfitting

Pre-pruning: limit growth with `max_depth`, `min_samples_leaf`, `min_samples_split`, or `max_leaf_nodes`.

Post-pruning: choose `ccp_alpha` to balance fit and tree size.

Model selection: tune these hyperparameters with cross-validation on development data.

Ensembles: random forests can reduce variance.

Advantages

- Easy to visualize and interpret for small trees
- Handles nonlinear decision boundaries and feature interactions
- Little need for numerical feature scaling
- Supports classification and regression
- Can model multi-output targets

Disadvantages

- Deep trees may overfit and generalize poorly
- Small changes in training data can change tree structure
- Greedy splitting does not guarantee a globally optimal tree
- Tree ensembles can improve stability and predictive performance
- Scikit-learn supports cost-complexity pruning using `ccp_alpha`

Additional Limitations

- Axis-aligned splits can be inefficient for some relationships
- Class imbalance may require appropriate metrics, weighting, or resampling
- Do not automatically balance every dataset: choose an approach based on the prediction objective
- Native categorical splitting is not supported by scikit-learn decision trees; encode categorical features appropriately